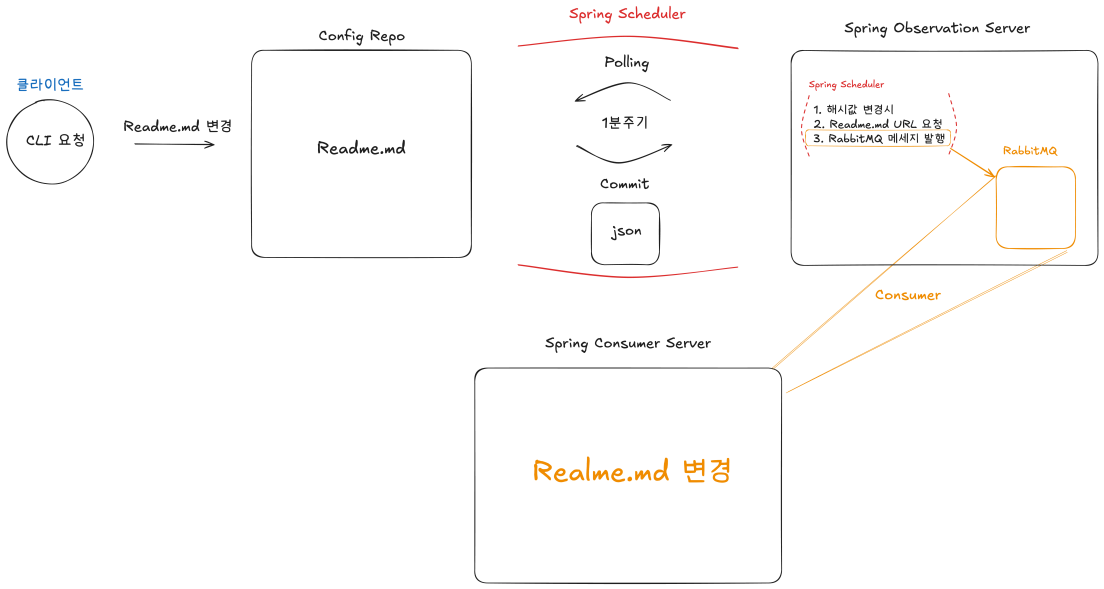
4. 실습

3. 구독서버 만들기

아래 최종 구조를 확인해보겠습니다.

💡 [최종구조]



CLI 요청

깃헙 config-readme에 CLI로 내용 변경을 커밋 후 푸쉬

```
vi readme.md
```

vi 에디터로 접근하여 내용을 변경합니다.

'# readme 테스트 1차' → '## readme 실습 1차'

이후 ':wq' 로 저장

```
nomadlab@nomadeulaeb-ui-MacBookPro config-readme % git add .
nomadlab@nomadeulaeb-ui-MacBookPro config-readme % git commit -m"readme 실습 1
차 "
[main fff5093] readme 실습 1차
 1 file changed, 1 insertion(+), 1 deletion(-)
nomadlab@nomadeulaeb-ui-MacBookPro config-readme % git log
commit fff5093394d753e20cc6d890c7020016228b0903 (HEAD -> main)
Author: kjh5848 <kjh5848@gmail.com>
Date:   Fri Oct 10 18:39:00 2025 +0900

    readme 실습 1차

commit cbcda934db14c5ea51a56c52e65c61068bb2258c (origin/main)
Author: kjh5848 <kjh5848@gmail.com>
Date:   Fri Oct 10 16:41:23 2025 +0900

    테스트 1차

commit d937305e1dbeb716514f64befb9cb478f9ba380f
Author: kjh5848 <kjh5848@gmail.com>
Date:   Fri Oct 10 10:56:05 2025 +0900

    first commit
nomadlab@nomadeulaeb-ui-MacBookPro config-readme %
```

커밋합니다. 로그를 보면 해시값이 fff5093394d753e20cc6d890c7020016228b0903 인걸 확인할 수 있습니다.

Observer Server를 처음 실행하면 메모이에 저장된 SHA의 값이 없을테니 무조건 변경을 감지하여 메세지를 발행합니다.

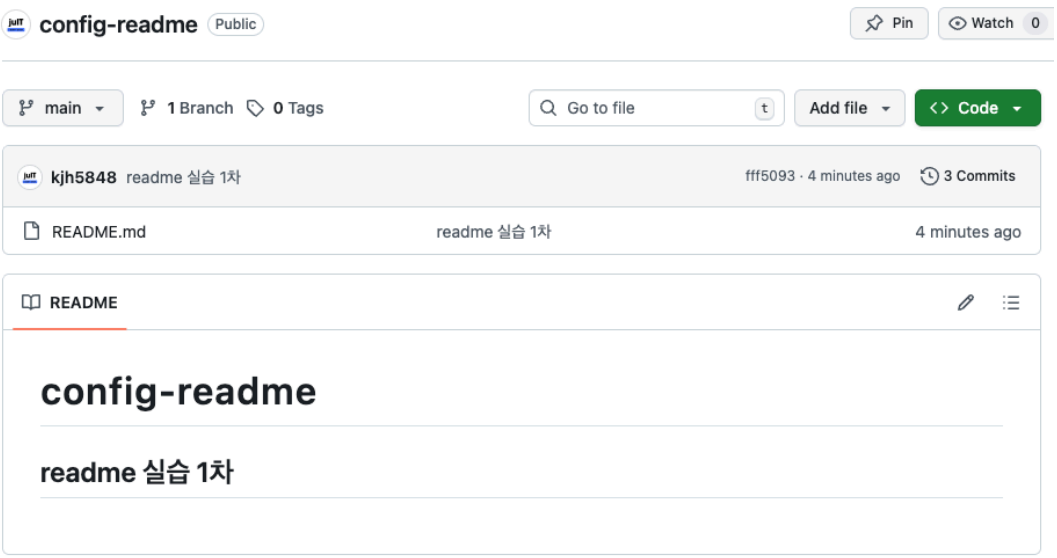
이후 2차,3차를 발행하여 백업파일이 올바르게 저장이 되는지 확인해보겠습니다.

이제 푸쉬를 하고 깃헙과 Observer Server, RabbitMQ, Consumer Server의 로그를 확인해보겠습니다.

```
nomadlab@nomadeulaeb-ui-MacBookPro config-readme % git push origin main
오브젝트 나열하는 중: 5, 완료.
오브젝트 개수 세는 중: 100% (5/5), 완료.
오브젝트 쓰는 중: 100% (3/3), 285 bytes | 285.00 KiB/s, 완료.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:kjh5848/config-readme.git
   cbcda93..fff5093  main -> main
nomadlab@nomadeulaeb-ui-MacBookPro config-readme %
```

---

1) 깃헙 config-readme



푸쉬 후 깃헙에 정상적으로 변경된걸 확인할 수 있습니다.

2) Observer Server

```
[폴링] 변경 없음 → SHA: cbcda934db14c5ea51a56c52e65c61068bb2258c
```

💡 로컬 환경에서는 GitHub와 직접적인 연결이 어렵기 때문에, 주기적으로 폴링(Polling)하여 변경 사항을 감지하고 Consumer 서버의 파일을 자동으로 갱신하는 방식으로 구현했습니다.
하지만 AWS EC2와 같은 클라우드 서버 환경에서는 GitHub Actions나 Webhook을 활용해 변경 이벤트를 실시간으로 감지할 수 있으므로, 별도의 폴링 로직 없이도 동일한 자동화가 가능합니다.